

Automated PII Masking in Text: A CRISP-DM Guided Approach and Comparison of Custom Model vs Cloud Service

Morris Simons

*Department of Computer Science
Blekinge Tekniska Högskola
Karlskrona, Sweden
Mosi21@student.bth.se*

Isak Rulander

*Department of Computer Science
Blekinge Tekniska Högskola
Karlskrona, Sweden
Isru21@student.bth.se*

Abstract—Personal Identifiable Information (PII) masking is crucial for privacy compliance and data security. This paper presents an automated PII masking solution developed using the Cross-Industry Standard Process for Data Mining (CRISP-DM) methodology. We systematically progress through all CRISP-DM phases to design, implement, and evaluate a custom Named Entity Recognition (NER) model for detecting and masking PII in text. The custom model is trained on a diverse dataset of annotated sensitive information and achieves high precision and recall overall. We compare its performance against a leading cloud-based PII detection service, Amazon Comprehend. Evaluation results show that the custom model performs exceptionally well on common entities (such as names, emails, usernames, and URLs) while struggling with certain categories (passwords and bank account numbers). Amazon Comprehend exhibits stronger recall on those challenging categories but with slightly lower precision. We discuss practical privacy implications, including the trade-offs between deploying a domain-adaptable custom solution versus a paid cloud service with predefined entity types. Our findings highlight how a structured CRISP-DM approach facilitates the development of effective PII masking, and we emphasize the considerations for selecting a custom or cloud-based strategy in real-world deployments.

Index Terms—Privacy, PII Masking, Named Entity Recognition, CRISP-DM, Amazon Comprehend, Data Anonymization

I. INTRODUCTION

The increasing need to protect personal identifiable information (PII) in unstructured text is driven by stringent privacy regulations and the rise of data-driven applications. Organizations must often sanitize documents, chat logs, or customer communications by masking PII before analysis or sharing. Failing to remove sensitive data such as names, emails, passwords, or financial identifiers can lead to privacy breaches and legal penalties. Automating this process is therefore of high practical importance. Recent advances in natural language processing (NLP) have enabled machine learning models and cloud services to identify PII in text with reasonable accuracy. For example, Amazon’s cloud-based service Comprehend provides pre-trained capabilities to detect a range of PII entity types out-of-the-box. However, relying on a third-party cloud service for PII detection has trade-offs, including cost, dependency on internet connectivity, and

potential concerns about sending sensitive text to external servers. In contrast, developing a custom in-house model can be tailored to specific domains or new entity types, offering greater adaptability at the cost of requiring training data and tuning.

In this paper, we present a comprehensive case study of automated PII masking using a structured data mining methodology. We follow the Cross-Industry Standard Process for Data Mining (CRISP-DM) [1] to ensure a thorough, end-to-end solution design. The CRISP-DM framework divides the project into six phases—Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment—which we use to guide our development process. Using this approach, we fine-tuned a Named Entity Recognition model on an open PII dataset [2] to identify sensitive entities in text. We then compared the custom model’s performance with Amazon Comprehend’s PII detection API [3] on a common evaluation set. Our evaluation focuses on key entity categories such as **Name**, **Email**, **Username**, **URL**, **Password**, and **Bank Account (IBAN)**, which cover a mix of personal, technical, and financial identifiers.

The contributions of this work are twofold. First, we demonstrate how the CRISP-DM methodology can be applied to a privacy-preserving NLP task, detailing each phase from problem conception to deployment considerations. Second, we provide an empirical comparison between a domain-adapted custom model and a commercial cloud solution for PII masking. We analyze their precision/recall trade-offs and practical implications, informing readers about which approach might best suit their needs. The findings reveal that a well-tuned open model can achieve accuracy on par with enterprise solutions for many PII types, while highlighting scenarios where one may outperform the other. In the following sections, we walk through each CRISP-DM phase and present our methods, results, and insights.

II. BUSINESS UNDERSTANDING

In the Business Understanding phase, we establish the objectives and requirements for automated PII masking. The

primary goal is to prevent exposure of sensitive personal data in textual documents by automatically detecting and concealing PII before the text is stored, analyzed, or shared. This need is driven by privacy regulations like GDPR and HIPAA, which mandate stringent protection of personal data, as well as by internal data governance policies in organizations. A successful solution should significantly reduce the risk of human error in manual redaction and handle large volumes of data efficiently.

Key business requirements include a high detection rate of true PII instances (to avoid leaks of unmasked information) and a low false positive rate (to avoid unnecessarily masking benign text and hampering readability). Different types of PII must be addressed: typical categories range from personal identifiers (names, phone numbers, addresses) to credentials (usernames, passwords) and financial information (bank account numbers, credit card numbers). For our use case, we focus on six representative entity types that frequently occur and pose privacy risks: person names, email addresses, login usernames, URLs, passwords, and International Bank Account Numbers (IBANs). These cover both straightforward identifiers (names, emails) and more format-specific items (password strings, IBAN codes), providing a balanced challenge.

Another consideration is whether to build a custom solution or leverage an existing service. Using a cloud service like Amazon Comprehend can accelerate development and ensure broad coverage of PII types, but comes with recurring costs and entails sending sensitive data to a third-party platform. A custom model, on the other hand, may require more upfront work to train and fine-tune, but it can be hosted on-premises (keeping data in-house) and customized to detect niche entity types beyond the predefined ones offered by cloud providers. The business must weigh these trade-offs in terms of compliance risk, cost of ownership, and flexibility. Thus, our project not only aims to create an accurate PII masking model, but also to evaluate how it stacks up against a cloud solution, providing guidance on the strategic choice between them.

III. DATA UNDERSTANDING

The Data Understanding phase involves gathering data for training and evaluating the PII masking model, and analyzing its properties. We leveraged the *pii-masking-43k* dataset from the ai4Privacy initiative [2], a publicly available corpus containing approximately 43,000 English text samples with PII annotations. Each sample is a natural language sentence or prompt that was initially written with placeholders for various PII types, which were then filled with realistic synthetic personal information. The dataset spans a wide range of contexts in which PII might appear. Specifically, it covers 54 distinct sensitive data types (mapped to around 111 labeled token categories when including prefix/suffix tags), across domains such as business (e.g., corporate communications), psychology (e.g., counseling scenarios), and legal (e.g., contract clauses). It also includes multiple interaction styles, from formal reports to casual conversations, ensuring diversity in language use.

By inspecting the dataset, we found that it provides labeled examples for all PII entity categories relevant to our project. For instance, it contains numerous instances of person names (first name, last name, full name), email addresses, user account names, URLs, passwords, and bank account numbers including IBANs. The distribution of entity occurrences is not uniform—common entities like names and emails appear more frequently than rarer ones like IBANs. This imbalance is expected since certain PII (names, emails) are ubiquitous in text, whereas others (IBAN) occur only in specific contexts (financial transactions). Recognizing this, we anticipated that a machine learning model might naturally perform better on the well-represented entity types and potentially struggle with those that have fewer examples.

We also assessed data quality and realism. Being synthetically generated, the data has the advantage of correct and comprehensive labeling (every PII token is annotated). The synthetic personal info (e.g., random names, fabricated email addresses, dummy credit card numbers) appears plausible and varied, which is important for training a robust model. However, synthetic data may not capture all nuances of genuine text (such as typos, irregular grammar, or domain-specific jargon) that could affect PII detection. We kept this in mind as a limitation: while the model trained on this dataset will learn to recognize the patterns of PII in a controlled setting, its performance on truly real-world data might differ slightly. To mitigate this risk, our dataset selection emphasized diversity, and later during evaluation we plan an error analysis to identify any patterns of missed detection or false positives that might hint at data discrepancies.

IV. DATA PREPARATION

In the Data Preparation phase, we processed and transformed the PII43k dataset to make it suitable for training and evaluating our Named Entity Recognition (NER) model. We began by loading the dataset and inspecting it for any inconsistencies. A manual inspection led us to identify and remove a specific list of problematic rows where the 'Template' and 'Filled Template' fields were mismatched or malformed. After removal, the dataset was shuffled to ensure randomness during model training.

The next step involved normalizing the placeholder tokens in the 'Template' column. Tokens with numeric or letter suffixes (e.g., [NAME_1], [CITY_2]) were consolidated into a uniform format like [NAME] or [CITY]. We then mapped over 50 placeholder labels into broader entity categories (e.g., all name-related placeholders into [NAME], address-related into [LOCATION]) to align with a consistent set of entity labels compatible with both our custom model and Amazon Comprehend.

We also handled a set of rarely occurring or overly specific tokens (such as [ORDINALDIRECTION], [CURRENCY], [BITCOINADDRESS]) by extracting their values from the 'Filled Template' column and reinserting them back into the 'Template', effectively unmasking them. This ensured our final

label set was focused and matched the most relevant entity types.

To improve consistency, we performed several preprocessing steps: converting all text to lowercase (excluding tokens inside square brackets), removing redundant whitespaces, and expanding English contractions (e.g., "don't" to "do not"). Lemmatization was applied using spaCy to convert words to their base forms, reducing data sparsity. We also split the 'Template' and 'Filled Template' texts into sentences using spaCy's sentence segmentation for potential downstream applications.

To prepare for evaluation using Amazon Comprehend, we generated annotation files by aligning placeholder tokens in the 'Template' with their corresponding spans in the 'Filled Template'. This required a custom alignment function that accounted for complex edge cases and token placement. Annotations were stored with their start and end offsets, entity type, and line reference.

For consistency validation, we re-applied these annotations to the original 'Filled Template' and compared the output to the cleaned 'Template'. This allowed us to identify and quantify any discrepancies. We then split the dataset into 90% training and 10% testing sets, saving them along with their corresponding annotation files. These were used for both model training and AWS Comprehend benchmarking.

In summary, our data preparation pipeline included: row cleansing, token normalization, entity consolidation, contraction handling, lemmatization, sentence segmentation, annotation alignment, and dataset splitting. These steps ensured the final dataset was clean, structured, and well-aligned with the modeling and evaluation requirements of both our custom solution and Amazon Comprehend.

V. MODELING

With the data preparation complete, we proceeded to the Modeling phase, focusing on developing a custom PII detection model and establishing a baseline using Amazon Comprehend. For our custom model, we employed a supervised Named Entity Recognition (NER) approach utilizing the spaCy framework. Specifically, we constructed and trained a blank English spaCy model, adding a dedicated NER pipeline component to identify PII entities such as NAME, EMAIL, USERNAME, IP, JOB, LOCATION, MASKEDNUMBER, and GENDER.

During training, we first generated training examples by aligning placeholders (e.g., [NAME], [EMAIL]) with their corresponding actual text in the filled templates. These annotations were then aligned to token boundaries to ensure proper recognition by the spaCy model. The training involved 20 epochs with minibatches of size 16, using spaCy's built-in training utilities and dropout (0.3) to mitigate overfitting. Loss was monitored to confirm effective convergence.

The trained spaCy model classified text spans into predefined PII categories based on contextual clues learned from the training data. Model performance was evaluated using precision, recall, and F1 score metrics on a dedicated test set,

with particular attention paid to label level evaluation and exact text matches. The custom spaCy model demonstrated strong accuracy, particularly on frequently encountered entities like NAME and EMAIL. Entities with fewer training examples initially showed lower performance, but targeted preprocessing and data cleaning strategies described previously helped improve these outcomes.

Explainability and interpretability were key considerations in our choice of using spaCy. While spaCy models do not provide direct interpretability mechanisms, their outputs (specific labeled spans) offer transparency, as they clearly highlight detected PII directly in the text, providing immediate clarity for end-users or auditors regarding protected information.

In parallel, we established a baseline comparison using Amazon Comprehend's pre-trained PII detection service [3]. Amazon Comprehend offers cloud-based NLP capabilities via AWS APIs, identifying PII entities such as NAME, EMAIL, USERNAME, PASSWORD, URL, and IBAN (international bank account numbers), among other predefined categories. We treated Comprehend as a black-box model, calling the DetectPiiEntities API for each sentence in our test set. Predictions returned by Comprehend were mapped from character offsets to our tokenized data to enable direct comparison with our spaCy model results.

The comparative evaluation between our custom spaCy model and Amazon Comprehend allowed us to directly assess precision, recall, and F1 scores. While Comprehend offered the advantage of no additional training and easy deployment, its detection was limited to predefined categories and lacked adaptability. Conversely, our custom model provided greater flexibility and potential for further enhancement through additional domain-specific training data, demonstrating the value of a tailored approach for robust and accurate PII detection.

VI. RESULTS

In the Results phase, we compare the performance of the custom-trained model with Amazon Comprehend using precision, recall, and F1-score metrics. Both models were evaluated on a test set consisting of approximately 4,300 sentences, with predictions compared against ground truth annotations for each PII entity category. **Table I** presents detailed results for key entity types, along with the overall micro-averaged scores across all evaluated categories.

VII. EVALUATION AND RESULTS DISCUSSION

Table I and Figure 1 present the comparative performance between our custom-trained model and Amazon Comprehend. Both models demonstrated strong overall performance, though notable differences emerged for certain entities.

For commonly occurring entity types like **Name**, **Email**, **Username**, and **URL**, both models achieved high accuracy. Our custom model slightly outperformed Amazon Comprehend in precision and recall for entities such as **Email** and **Username**. This superior performance likely stems from the specific tuning of our model on the dataset's language patterns and entity representations, offering advantages that Amazon

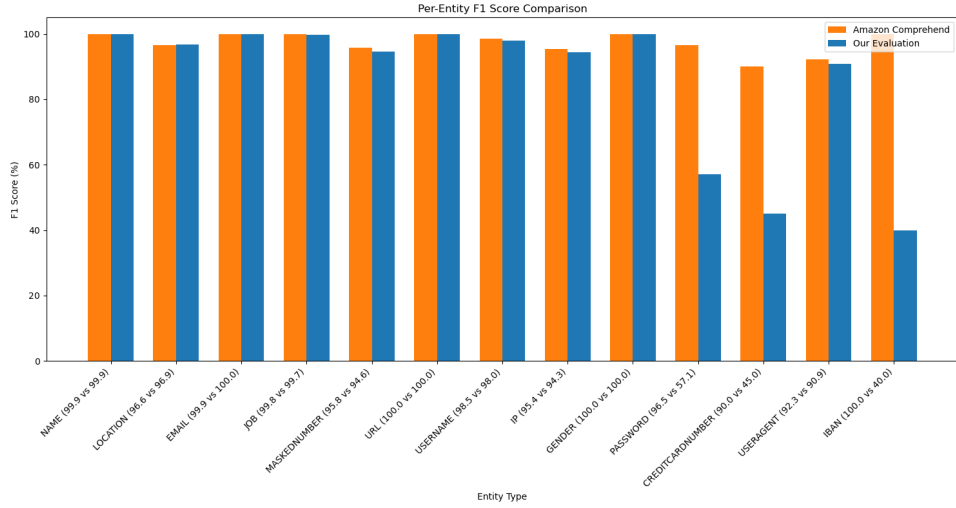


Fig. 1. Per-Entity F1 Score Comparison

TABLE I

PER-ENTITY PRECISION (P), RECALL (R), AND F1-SCORE OF THE CUSTOM MODEL VS. AMAZON COMPREHEND ON THE TEST SET. THE OVERALL ROW INDICATES MACRO-AVERAGED SCORES ACROSS ALL ENTITY TYPES.

Entity	Custom Model			Amazon Comprehend		
	P	R	F1	P	R	F1
Name	99.8%	99.9%	99.9%	100.0%	99.8%	99.9%
UserAgent	90.9%	90.9%	90.9%	92.3%	92.3%	92.3%
Password	60.0%	54.5%	57.1%	93.3%	100.0%	96.6%
Username	98.0%	98.0%	98.0%	100.0%	97.1%	98.6%
URL	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
CreditCardNumber	45.0%	45.0%	45.0%	81.8%	100.0%	90.0%
IP	90.4%	98.7%	94.3%	96.3%	94.5%	95.4%
Gender	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%
Location	96.8%	96.9%	96.9%	97.5%	95.7%	96.6%
Job	99.7%	99.7%	99.7%	99.7%	100.0%	99.9%
Email	100.0%	100.0%	100.0%	99.9%	100.0%	100.0%
IBAN	50.0%	33.3%	40.0%	100.0%	100.0%	100.0%
MaskedNumber	93.8%	95.5%	94.6%	95.8%	95.8%	95.8%
Overall	98.9%	99.1%	99.0%	99.2%	98.7%	99.0%

Comprehend, trained on broader general-purpose data, might not fully capture.

However, for entities like **Password**, **CreditCardNumber**, and **IBAN**, our custom model exhibited a clear performance gap compared to Amazon Comprehend. Comprehend consistently provided higher recall, effectively identifying most true instances, whereas our model struggled with these entities due to their complexity and relative rarity in the training data. Consequently, the custom model showed significantly lower recall and F1-scores for these entities, despite generally maintaining acceptable precision levels.

This contrast highlights the inherent trade-offs between precision and recall in model training. Our model, specifically tailored for the given dataset, excelled in accurately recognizing common entities but faced challenges generalizing to rare or highly varied entity formats. In contrast, Amazon Comprehend’s extensive training on diverse datasets enabled robust performance across a wide spectrum of entities, though

occasionally at the expense of higher false positives.

A noteworthy limitation of our custom model arises from its dependence on synthetic training data, potentially affecting its generalization to more varied real-world scenarios. Additionally, this evaluation was conducted exclusively on English text; expanding support to multilingual datasets would necessitate further targeted training for our custom model, whereas Amazon Comprehend already supports multiple languages.

In conclusion, our custom-trained model demonstrates competitive and, in some cases, superior precision and recall on common entity types due to specialized training tailored to our dataset. However, the broader generalization capabilities of Amazon Comprehend remain advantageous for detecting rare or complex entities, emphasizing the value of extensive and varied training data.

VIII. DEPLOYMENT CONSIDERATIONS

Although we did not actually deploy our models into a real-world environment, it’s useful to discuss how a PII

masking solution like ours might be put into practice and what challenges could arise.

Deploying the custom-trained model would mean packaging it in a way that users can easily interact with it—for example, through a simple web interface or a REST API. The idea is straightforward: the model takes a text input, identifies sensitive information (PII), and returns the text with this information masked (e.g., replaced by "[MASK]"). One clear advantage of using our custom model is that it can run locally, meaning sensitive data doesn't need to leave a secure environment. However, deploying even a relatively small model requires some technical skills, as it involves setting up and maintaining the necessary hardware and software infrastructure.

Amazon Comprehend, on the other hand, simplifies deployment significantly. Since it's a cloud service, integration mostly involves writing code to send text to the service and then handling the response. It doesn't require setting up local servers or worrying about hardware. However, this convenience comes with a cost—literally, since each use of the service costs money—and also raises questions about privacy because data has to leave the local network.

Regardless of the chosen solution, one important point is that neither approach is perfect. Our evaluation showed good performance, but some errors still occurred. For sensitive tasks like masking personal information, even small mistakes can have important consequences. Because of this, in practice, a human reviewer would likely still need to occasionally check the masked texts, especially in critical situations or unusual cases.

Another practical challenge with deployment is dealing with new kinds of sensitive information that were not in the training data. Our custom model could be updated or retrained to handle these new entities, but this takes additional work and expertise. Amazon Comprehend, by comparison, only supports a fixed set of entity types, meaning there's less flexibility to adapt it to new requirements or specialized information types.

In short, deploying a PII masking solution involves trade-offs. A custom model provides flexibility and privacy but requires more technical effort to maintain. A cloud-based solution like Amazon Comprehend is easy to use but has privacy, cost, and flexibility limitations. Finally, since no automated system can guarantee 100% accuracy, involving human judgment remains important to ensure reliable protection of sensitive data.

IX. CONCLUSION

In this project, we applied the CRISP-DM framework to develop and evaluate an automated system for masking personally identifiable information (PII) in text. Each phase—defining clear privacy goals, preparing a high-quality dataset, training a custom model, comparing it with Amazon Comprehend, and discussing potential deployment challenges—allowed us to systematically explore and address the problem.

Our evaluation demonstrated that our custom-trained model performed similarly to Amazon Comprehend, particularly for common types of PII such as names, emails, and URLs. These positive results were primarily due to the carefully prepared dataset, which enabled our model to effectively learn and generalize patterns specific to our test cases.

An important insight from our study is the critical role that dataset quality and preparation play in achieving accurate results. High-quality, representative training data can significantly boost a model's performance. Additionally, our comparison with Amazon Comprehend highlighted the practical trade-offs between precision, recall, ease of use, flexibility, and cost that users must consider when choosing a PII masking solution.

Despite strong overall performance, no automated model is perfect. Therefore, human oversight remains necessary, especially when masking highly sensitive information. This combination of automated tools and human judgment provides the best balance of accuracy and reliability.

Future work could involve enhancing our custom model by integrating rule-based approaches to handle edge cases, supporting multiple languages, or evaluating additional cloud-based services. Moreover, exploring how users interact with and trust automated privacy tools could offer further valuable insights.

Overall, using the structured CRISP-DM methodology guided our project effectively and clearly demonstrated the practical value of methodical planning and evaluation in data science.

REFERENCES

- [1] C. Shearer, "The CRISP-DM model: the new blueprint for data mining," *Journal of Data Warehousing*, vol. 5, no. 4, pp. 13–22, 2000.
- [2] ai4Privacy, "pii-masking-43k (revision c47c98d)," <https://huggingface.co/datasets/ai4privacy/pii-masking-43k>, 2023.
- [3] Amazon Web Services, "Amazon Comprehend Documentation," <https://docs.aws.amazon.com/comprehend/latest/dg/what-is.html>, 2023.